

【SOP】 Air_Sim 仿真搭建与强化学习零基础教程

一、仿真快速使用

1.项目简洁

把 Air_Sim 想成自动运行的 3x3 救援棋盘。红方在网格中巡逻，蓝方在开局部署结束后分批出现。红方能在同一网格内攻击蓝方；当一个已出现蓝方所在格没有红方实际占据时，系统构造跨格增援问题。此时页面可请求 Exact 或 RL 决定哪些红方支援哪些蓝方。这不是预先录好的动画。每个单位都有坐标、状态、指令和时间；出生、移动、碰撞与增援都按规则计算并记录。

当前事实	当前值	初学者怎样理解
默认规划器	默认 Exact	不切换时使用确定性精确规划
可选模型	ppo-v3	页面异步加载的强化学习模型
模型种子	seed 212	当前导出模型来自该训练 run
输入	17 维	每个合法红蓝候选组合用 17 个数描述
可训练参数	26,050	神经网络中可学习的权重和偏置总数
正式评估	600 条	480 条 synthetic、120 条 simulation
当前保护	回退 Exact	模型异常时仍执行可解释规划



创建 RL 环境	<code>py -3.12 -m venv .venv</code>	<code>.venv\Scripts\python.exe</code> 存在	别处激活的环境不能替代它
安装 RL 依赖	<code>./venv/Scripts/python.exe -m pip install -r requirements-rl.txt</code>	无 traceback	PyTorch 下载较大，耗时不等于卡死

```
Plain Text
npm.cmd ci
py -3.12 -m venv .venv
.\venv\Scripts\python.exe -m pip install --upgrade pip
.\venv\Scripts\python.exe -m pip install -r requirements-rl.txt
```

requirements-rl.txt 安装 NumPy、pytest、TensorBoard 和 PyTorch。这里不要求 CUDA；当前基线是 CPU 环境，`torch.cuda.is_available()` 显示 False 也能学习、测试和训练，只是训练会慢一些。

3. 启动页面

在项目根目录启动本地静态服务器。不要双击 `index.html`，因为配置和模型需要通过 HTTP 正确加载。**Node** 在这里仅发送静态文件，不是业务后端：没有账号、数据库、业务 API 或服务器端仿真。

```
Plain Text
npm.cmd run dev
```

终端应显示 `http://127.0.0.1:4173/`。保持该终端运行，在浏览器打开这个地址。页面成功时，顶部配置不再显示加载中，模型区域显示 `ppo-v3` 已就绪或可用，中央出现九个网格。停止服务器时回到终端按 `Ctrl+C`。

区域	先看什么	它回答的问题
顶部状态栏	时间、暂停、倍速、算法	仿真是否推进，当前请求什么规划
左侧网格属性	地势、天气、烈度	每格当前环境与蓝方数量
中央态势棋盘	红蓝位置与轨迹	单位在哪里、正在做什么

右侧选中单位	ID、状态、位置、目标	某一单位为什么移动或攻击
右侧增援调度	当前计划与历史	请求了什么、实际用了什么、是否回退

4.仿真运行

1. 页面启动后运行几秒并点击暂停。点击一个红方，读取它的 ID、状态、当前位置和目标网格。
2. 点击继续，观察蓝方不会开局立即全部出现。红方完成初始部署后，出生门打开，5 秒后才出现第一批蓝方。
3. 等右侧“增援调度”出现历史记录，展开一项计划。Exact 模式下，requested 和 used 都应表明 Exact。
4. 切换到 RL，重置后再运行。若出现 RL 计划，核对实际算法和 ppo-v3；若出现 fallback，表示安全机制执行了 Exact，不是页面失效。

5.控制按钮说明

控件	它改变什么	它不改变什么
暂停/继续	是否推进仿真时间	已有规则和单位状态
重置	初始单位、种子和控件	配置文件和浏览器模型
0.5x/1x/2x	现实时间换算仿真时间的速度	同一时刻事件的顺序
Exact/RL	下一次增援的请求算法	合法性、来源容量和回退规则

你选择 RL 只表示“请求 RL”；模型尚未就绪、输入不合法或推理异常时，历史仍会记录实际使用 Exact。做对比时先重置，再只比较同一轮的第一项计划，避免之前的战斗和调度改变后续局面。第一次练习只需要得到两个可观察证据：页面能持续推进时间，计划历史能说明本次请求和实际执行的算法。不要把“看见红方移动”当作已经验证了 RL；只有在历史中同时看到 used、modelVersion 和可能的 fallback 原因，才知道调度路径实际发生了什么。也不要因为一次没有产生跨格增援就反复刷新页面。默认场景中的出生与运动按固定时间推进，耐心运行、暂停观察并在需要时重置，才是可

比较的操作方式。

读一条增援计划时，建议按固定顺序看五件事。先看 `requested`，它表示你在界面上请求 `Exact` 还是 `RL`；再看 `used`，它才是本次真正执行的算法。第三看 `modelVersion`，只有实际使用 `RL` 时，当前版本才应与 `ppo-v3` 对应。第四看覆盖数和总距离：覆盖数优先，覆盖相同后距离才有比较意义。最后看 `fallbackReason`；它为空表示没有回退，有内容则应把这条计划记为“请求 `RL`、实际 `Exact`”，而不是记成 `RL` 成果。

一条计划从“当前”进入“历史”也不是被删除。当前页签表示仍有分配在执行；历史页签保留已经完成或含取消项的计划。新手做记录时最好抄下计划 ID、请求算法、实际算法、覆盖数、总距离和回退原因。这样暂停、继续或切换页签后，仍能解释自己看到的结果，而不会只凭一张截图猜测。当你想把观察变成一个小实验时，准备一张纸或文本记录三列：重置后的算法模式、第一项计划的覆盖数、计划总距离。先记录 `Exact`，再记录 `RL`。两次记录的目的不是宣布某一次谁“赢了”，而是练习区分基线、模型、回退和受同一业务规则约束的结果。后续章节会解释为什么不同时间点的计划不能随意横向比较。

一个合格的首次对比应按同一流程完成。先等模型显示 `ppo-v3` 就绪，选择 `Exact` 并重置；运行到第一条增援计划进入历史，立即暂停并记下仿真时间、覆盖数和总距离。然后选择 `RL`、再次重置，从相同初始状态运行到第一条计划并记录同样字段。若 `RL` 发生回退，这轮结果只能证明回退路径可用，不能作为 `PPO` 距离数据。两轮的计划时间或需求集合不一致时，也不要硬比距离。

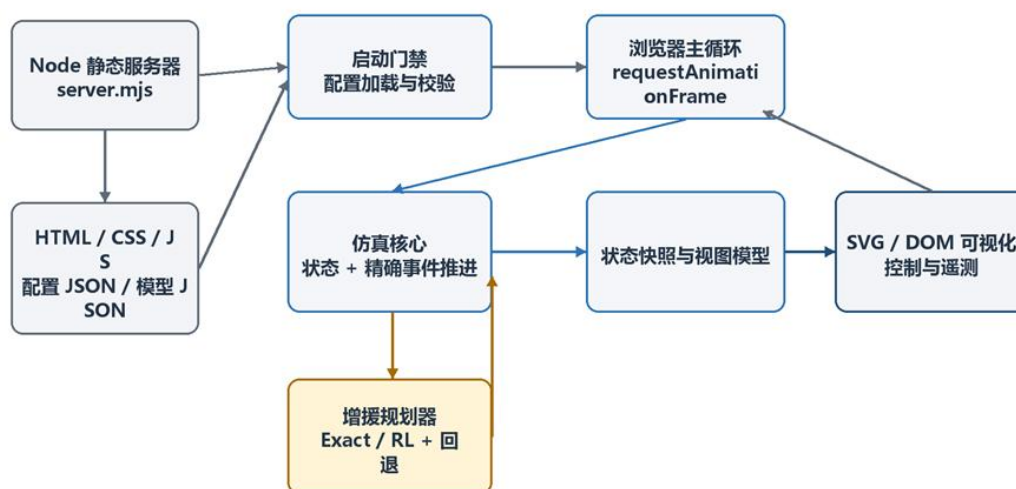
即使两轮问题一致，一次结果也只是观察，不是统计结论。`Exact` 的距离应作为该冻结问题的最优基线；`RL` 距离相同表示它找到了同距离方案，略长表示它仍合法但不是距离最优。覆盖数不同通常说明比较对象、时间点或回退记录读错了，因为两种规划器共享固定最大覆盖配额。真正评价模型要使用后文的 600 条固定数据，而不是挑一张看起来漂亮的页面截图。

完成观察后，可以回答四个自检问题：我比较的是不是同一初始状态的第一项计划；`RL` 是否真的被 `used`；两边覆盖数是否相同；我报告的是单例现象还是正式评估。能回答这四项，才算真正会使用界面，而不是只会点击按钮。

二、仿真搭建

1. 仿真组成部分

浏览器负责加载配置、创建单位、推进仿真、选择规划器和渲染 `SVG`；`server.mjs` 只发送文件。配置下载、解析和契约校验都通过后，程序才创建规划器、仿真对象和动画循环。浏览器把现实时间换成仿真时间，再用 `advanceTo()` 把状态精确推进到下一事件。



关键边界：Node 只提供静态资源；运动、事件、调度和渲染均在同一浏览器页面内存中完成。

图 2 总体结构。

模块	职责	初学者应抓住的点
config/grid-attributes.json	九格坐标与属性契约	配置错误时系统不会开始仿真
src/simulation.js	时钟和事件总导演	决定事件按什么时间处理
src/combat.js	本地红蓝配对	是稳定最近距离贪心，不是 Exact
src/reinforcement-problem.js	增援问题和合法候选	先由规则确定能选什么
src/reinforcement-exact.js	精确增援解	提供可验证的基线
src/reinforcement.js	Exact/RL 开关和回退	记录 requested、used 和 fallback

2. 网格、坐标和单位状态介绍

棋盘是 3 列乘 3 行，但单位坐标是连续实数，范围从 (0,0) 到 (3,3)。例如 (0.37,2.14) 不在离散点上，仍属于一个网格。默认有 20 个红方 R-01 到 R-20，24 个蓝方 B-01

到 B-24；蓝方最多可达 81 个，因为每格有 9 个固定槽位。

默认数字	值	为什么重要
红方数	20	初始目标分布覆盖全部九格
蓝方预算	24	不会无限生成新的 ID
每格蓝方槽位	9	最大蓝方数为 81
仿真种子	20260713	重置后可重复出生序列
开门后首次出生	5 秒	不是仿真开始后固定 5 秒
后续出生间隔	10 秒	每批总量为 1 到 3
地势权重	3:1:1	平原:湖泊:山地，只影响蓝方目标生成

字段	它回答的问题	主要用途
targetGrid	指令让我去哪里？	本地战斗候选和目标显示
currentGrid	当前坐标按公式落在哪格？	普通位置展示
occupancyGrid	越界后物理上已占据哪格？	增援需求和来源判断

Plain Text

R-01 正在从 (1,0) 移向 (2,1)
targetGrid = (2,1) 指令目标
currentGrid = (1,0) 坐标当前所在
occupancyGrid = (1,0) 已确认物理占据

红方的核心状态是转场、巡逻、攻击和死亡；蓝方的核心状态是等待、出现、交战和死亡。waiting 蓝方已有编号但不在棋盘上，只有 active 蓝方参与本地战斗或成为增援

需求。红方巡逻时才可能作为跨格增援供给者；正在攻击、转场、死亡或已有执行中增援的红方不能被重复调派。

可以把状态理解成“当前是否有资格参加某项规则”，而不只是界面标签：

- 红方转场表示正在执行目标指令，还没有稳定占据目的格；到达后进入巡逻，才可能成为跨格支援来源。
- 红方攻击表示已经与一个本地蓝方配对，同一时刻不能再被另一场战斗或增援重复使用。
- 蓝方等待表示槽位已经存在但尚未出生；出现后成为 **active**，才会显示、参加战斗或形成增援需求。
- 红蓝进入交战后按项目规则同归于尽，死亡单位仍保留在名册和历史中，但不再计入可用资源。

3.出生、战斗与增援介绍

初始红方全部到达部署目标或被新指令豁免后，出生门打开。第一批等待 5 秒，之后每 10 秒生成一批，直至 24 个蓝方 ID 用完。地势 3:1:1 只影响蓝方被分配到哪个地势格的概率；天气和烈度不改变当前项目的运动或战斗规则。本地战斗看 **targetGrid**：存活、未攻击且有目标的红方，若目标格与 **active** 蓝方所在格相同，就能进入候选。候选按距离、红方 ID、蓝方 ID 的稳定顺序扫描，一对红蓝配上后双方都不再可配。这是局部贪心匹配，不应称为全局最优。跨格增援看 **occupancyGrid**：一个 **active** 蓝方所在格没有存活红方物理占据时，才是需求。来自其他格的红方必须处于巡逻、没有本地攻击，也不能让来源格被抽空。若来源格有 n 个可用巡逻红方，最多调出 $n-1$ 个，始终至少留一名。

4.仿真运行结果一致性说明

确定性不等于“画面帧率永远相同”，而是指给定相同配置、种子和指令，不论浏览器把时间切成大步、小步还是约 60 Hz 帧，指定仿真时间的业务状态都应一致。实现依赖固定伪随机数、解析移动与碰撞时间、同一时刻固定处理顺序。



图 3 事件处理顺序。

Plain Text

找下一最早事件

- > 把所有单位精确积分到该时间
- > 处理目标、碰撞、出生、到达、越界等同时刻事件
- > 业务状态稳定后，再判断是否需要增援

暂停只是不再推进时间；0.5x、1x、2x 改变现实秒到仿真秒的比例，不改变同一时间戳的处理顺序。页面没有胜负判定、自动终局或持久化，持续运行本身不是故障。

5.把 Exact 当作正确性基线

Exact 接收当前时刻冻结的增援问题，在所有合法红蓝组合中求解。它不是训练模型，不预测未来，也不会绕过来源容量或状态约束。目标有严格优先级：先最大覆盖，再最短总距离，最后用稳定字典序打破完全相同的方案。

Exact 的目标	具体含义	不允许的误解
最大覆盖	尽量分配更多需求蓝方	不能为了少飞而少救一

		个
最短总距离	覆盖数相同才比较总距离	不是每次挑眼前最近一对
稳定字典序	前两项相同才按 ID 稳定选择	不是随机破平

Exact 增援规划的最小费用最大流网络

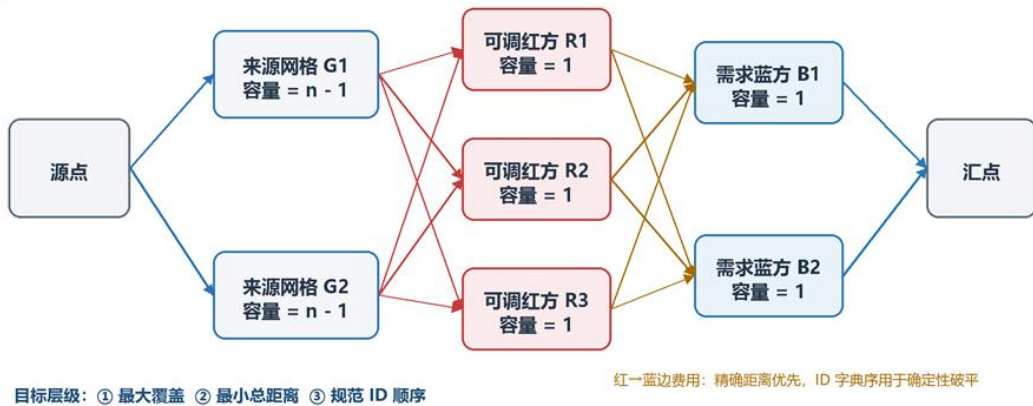


图 4 Exact 的最小费用最大流示意。

Plain Text
 来源格 --容量 $n-1$ --> 可调红方 --容量 1--> 需求蓝方 --容量 1--> 汇点
 \-----费用: 红蓝直线距离-----/
 优先: 最大流 (最大覆盖) -> 最小费用 (最短总距离) -> 稳定 ID 顺序

这就是 Exact 能作 RL 老师和评估基准的原因：两者面对同一个合法候选集合，差别只在选择方式。Exact 给出基线答案，RL 学习在约束内尽量接近它的距离表现。

看一个简化例子。假设来源 A 可调 1 名红方，来源 B 也可调 1 名；需求蓝方有 X、Y。A 到 X 很近，B 到 X 也不远，但只有 A 能以合理距离到达 Y。若只贪心地先把 A 派给 X，B 再去 Y 可能产生很长路线；Exact 会同时比较完整的两项分配，可能选择 B 到 X、A 到 Y，使覆盖数仍为 2 且总距离更短。

这个例子说明“每一步最近”不等于“整轮总距离最短”。Exact 用网络流一次考虑红方唯一性、蓝方唯一性、来源容量和所有候选距离。它也不会因为某条路线很长就牺牲覆盖：只要还能合法多覆盖一个需求，就先完成最大覆盖，再在同覆盖数方案中比较距离。

规划结果生成后，仿真还要把抽象配对变成实际指令。计划会记录来源格、目标蓝方、单项距离、总距离、请求/实际算法和执行状态；红方进入执行中增援后，本轮不能再被选。Exact 的“最优”只针对该冻结问题，不代表它预知后续出生、战斗或未来指令。

冻结快照是理解结果的关键。规划器看到的是某个确定时刻的需求蓝方、来源容量、可调红方、候选距离和当前执行占用；规划完成后，仿真时间继续推进，单位会移动、交战或完成计划。后来再看棋盘时，画面可能已不同于求解瞬间，但历史计划仍应解释当时为什么产生该分配。不要拿“现在的位置”重新计算旧计划并断言它错误。

同一时刻的处理顺序也保证快照一致：系统先把单位推进到事件时间并完成该时刻已有事件，再构造增援需求。这样蓝方刚出生、红方刚到达或一场本地战斗刚结束时，不会因浏览器帧切分不同而让规划器看到半更新状态。**Exact** 和 **RL** 都从同一问题对象开始，因此算法比较不需要各自维护一套业务规则。

三、强化学习

1.把强化学习映射到项目

强化学习不是让模型控制所有单位的连续运动，也不是训练整场红蓝对抗的胜负。这里每个 **episode** 从一个跨格增援问题开始，策略连续选择若干红蓝配对，直到固定配额完成。环境负责规则，策略只负责在合法候选中排序。

RL 概念	项目中的对应物	一句话理解
环境	增援问题和合法性规则	告诉模型当前能做什么
观测	17 维特征和全局上下文	告诉模型局面长什么样
动作	选择一个红方到蓝方候选配对	一步只选一条合法边
奖励	归一化负距离	路越短，惩罚越小
策略/Actor	为候选输出 logit	给每个可选配对打分
Critic	估计剩余回报	辅助训练，不直接下命令

固定配额等于“需求蓝方数”和“所有来源格可捐容量”的较小值。合法动作掩码排除已用红方、已覆盖蓝方、容量耗尽来源格和已完成配额的问题。因此 **RL** 不能通过少派单位投机；最大可能覆盖由规则和配额保证，模型主要学习如何缩短总距离。

2.网络输入说明

每一个合法红蓝候选组合都有 17 维，而不是整盘棋只有 17 个数字。前 7 维描述连续坐标与几何关系，中间 8 维描述来源/目标网格和剩余资源，最后 2 维提供稳定 ID 次序。这样模型既看见局部距离，也看见当前可调资源。

组别	维数	代表字段	为什么需要
连续坐标与几何	7	红/蓝 x、y、差值、距离	知道候选有多远、方向怎样
来源格与目标格	4	source/target col、row	知道从哪格支援到哪格
动态资源与进度	4	容量、未用红方、需求、配额	知道这一轮还能派多少
稳定 ID 排序	2	attacker/defender rank	几何近似时保持一致选择

Plain Text

候选 R-03 -> B-11

```
features = [红方坐标, 蓝方坐标, delta, distance,
            来源格, 目标格, 剩余容量, 剩余需求, ID rank]
mask      = [只保留仍合法的候选动作]
```

一条真实候选的 17 维归一化结果可能是：

```
[0.040506, 0.744596, 0.964909, 0.579011,
0.924404, -0.165585, 0.664056,
0.000000, 1.000000, 1.000000, 0.500000,
0.150000, 0.200000, 0.913580, 0.450000,
0.000000, 0.000000]
```

ID rank 不是战斗力排名。它只是把 R-01 到 R-20、B-01 到 B-81 的稳定次序告诉模型，帮助相同输入在不同运行中得到稳定结果。

这些值多数被缩放到大约 0 到 1，方向差值可以为负。归一化让坐标、距离、容量和 ID 次序处在相近数值范围，减少某一类大数值仅因尺度就主导网络。示例只是一个合法候选，不表示 **Exact** 最终一定选了它；一次决策会同时编码当时所有合法候选，再由全局上下文比较它们。

当前特征里没有地势、天气或烈度类别。这并非遗漏当前规则：地势只影响蓝方目标格的生成概率，天气只展示，烈度是蓝方数量的显示结果，它们都不改变当前移动代价、战斗或增援距离。未来如果地势开始改变速度或可达性，就必须升级特征 schema、重新生成数据并训练，不能静默沿用这 17 维。

2.调度方案生成

PPO-v3 先把每个候选编码为 64 个数，再用 mean/max 池化汇总整组候选。Actor 为每个候选产生分数，Critic 对整个局面估计价值。部署时不随机采样，而是从掩码保留的候选中取分数最高者，即 masked argmax。

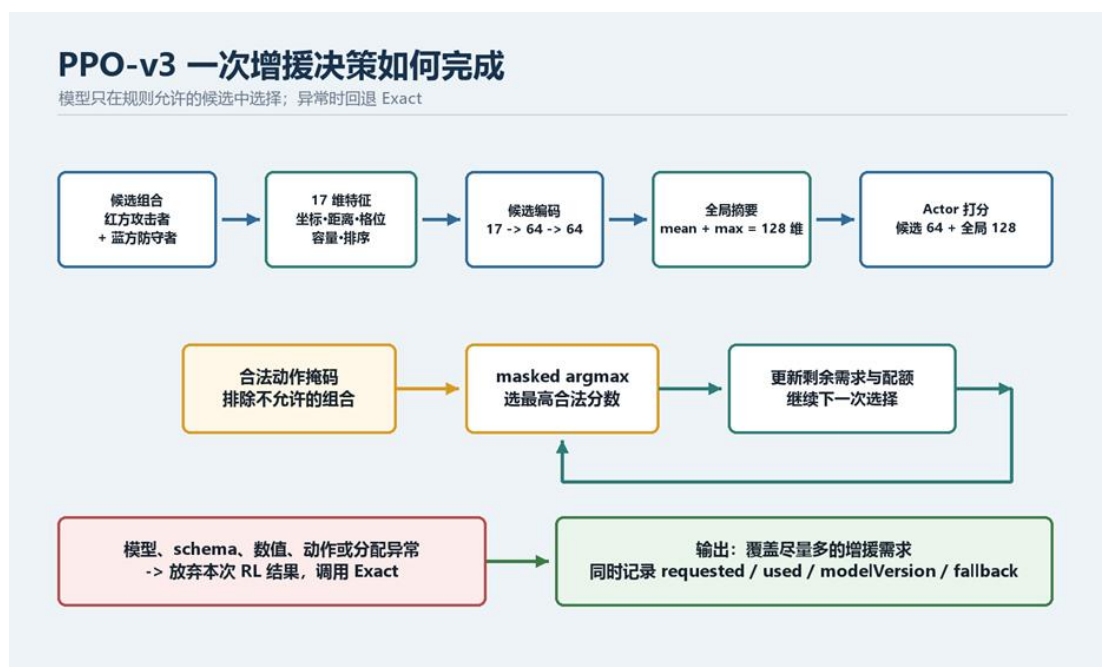


图 5 一次 RL 决策

网络部件	输入/输出	作用
候选层 17->64	1,152 参数	开始将候选变成可学习表示
候选层 64->64	4,160 参数	形成每个候选的 64 维编码
全局池化	多候选 -> mean/max 上下文	让一个候选看见整轮局面
Actor 192->64->1	12,417 参数	为每个候选输出一个 logit
Critic 128->64->1	8,321 参数	为全局局面输出一个 value
合计	26,050 参数	当前浏览器模型的参数

		量
--	--	---

Plain Text

```
logits = Actor(候选编码, mean_context, max_context)
masked_logits = 对非法候选设为不可选
action = argmax(masked_logits)          # 部署: masked argmax
若加载、输入或推理失败: action = Exact(problem) # 回退 Exact
```

奖励是归一化负距离。距离越长，奖励越负；一整轮累计奖励越接近 0，通常表示总增援距离越短。奖励不额外奖励覆盖，因为固定配额和合法动作掩码已经强制完成可达到的覆盖数。

参数量的算法是“输入数乘输出数，再加每个输出的偏置”。例如第一层 $17 \times 64 + 64 = 1,152$ 。26,050 不是 26,050 条人工规则，而是训练会调整的数字总数。
mean 池化描述候选的整体水平，**max** 池化保留每一维最突出的信号；两者拼成 128 维全局上下文，使 **Actor** 评分时不只看一条边。

训练阶段与部署阶段也要分开。**PPO rollout** 会在合法动作分布中采样，以探索不同完整序列；验证、评估和浏览器使用确定性的 **masked argmax**。相同问题、相同权重和相同候选规范顺序下，部署应产生相同选择序列。**Critic** 只在训练时帮助估计回报，浏览器真正下达配对的是 **Actor** 的候选分数。

3.BC、PPO 说明

训练先由 **Exact** 专家 **JSONL** 给出完整选择序列。行为克隆 **BC** 把专家动作当监督答案，让模型先学会近距离的合理排序；**PPO** 再让策略按自身概率产生完整合法轨迹，通过回报做小步更新。**best-*.pt** 是验证集上合法、全覆盖且 **mean gap** 更小的 **checkpoint**；**last-*.pt** 只是最后一轮，不保证更好。

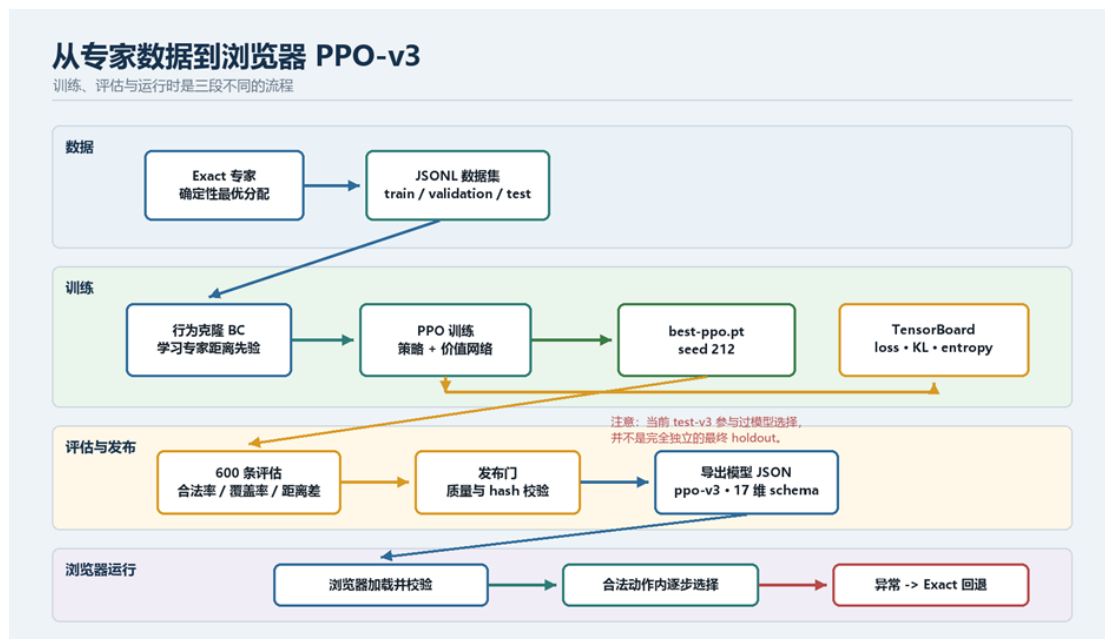


图 6 训练到浏览器的路径

当前 PPO-v3 证据	数值	正确结论
来源 checkpoint	ppo-v3-212/best-ppo.pt	checkpoint 类型为 ppo
最佳更新	18 / 20	不是最后一轮的默认替代品
测试记录	600 条	synthetic 480、simulation 120
legality / coverage	100% / 100%	固定测试集没有非法或未完成序列
overall mean gap	1.2236%	相对 Exact 的逐记录平均距离差
overall P95 gap	4.7040%	大约 95% 非零 Exact 样本不超过该差距

Plain Text

Exact 专家 JSONL -> BC 初始化 -> PPO 更新 -> best-ppo.pt
 best-ppo.pt -> 600 条评估 -> 通过门禁 -> ppo-v3 浏览器 JSON
 训练日志 -> TensorBoard

当前 ppo-v3 已通过项目门禁，但这不是“RL 超过 Exact”的证明，Exact 仍保有精确最优性保证。普通 BC/PPO 比较中，BC-v3 略优；当前测试集也不是完全独立的最终 holdout。把这些限制写清楚，比只报告漂亮数字更可靠。

mean gap 的计算单位是“每条记录”。对一条 Exact 总距离大于 0 的记录，先计算 (策略总距离 - Exact 总距离) / Exact 总距离，再把负值截为 0；overall mean 是这些记录 gap 的平均。它不是“平均策略距离除以平均 Exact 距离”，也不是绝对多飞了多少坐标单位。0.012236... 是比例，换成百分数约为 1.2236%，不能读成 0.012%。

P95 gap 用来观察长尾：约 95% 的非零 Exact 样本，其相对差距不高于 4.7040%。mean 小而 P95 很大，可能表示多数样本接近 Exact、少数样本明显较差；所以门禁同时检查二者。合法率和覆盖率则是硬条件，先保证每条完整选择序列遵守规则并完成配额，才有资格比较距离。

当前评估按 overall、synthetic、simulation 三组检查，三组都要通过。PPO-v3 的 600 条记录来自当前测试数据，其中 synthetic 480 条、simulation 120 条。它们覆盖项目定义的问题分布，但没有构成模型冻结后完全未参与选择的最终独立 holdout。因此正确表述是“在当前 600 条项目评估上通过门禁”，而不是“未来所有场景平均只差 1.2236%”。TensorBoard 最值得先看六项：validation/legality、validation/coverage、validation/mean_distance_gap、validation/p95_distance_gap、训练损失和平均 episode reward。前两项必须保持 1.0；gap 越低越接近 Exact；曲线波动不自动等于模型可部署，仍需经过评估门。

第一次打开 TensorBoard，可以按下面顺序读，而不必一次理解所有曲线：

- 先确认 legality 和 coverage 始终为 1.0；任何下降都比 loss 是否好看更重要。
- 再比较 validation mean gap 和 P95 gap。mean 反映平均水平，P95 帮助发现少数明显较差的长尾样本。
- 然后看平均 episode reward 是否在相近问题分布下更接近 0；不同数据规模之间不要直接横比。
- 最后才看 Actor/Critic loss。PPO 的策略损失没有“越低必然越好”的单一解释，必须回到验证指标。

把训练曲线和发布结论分开理解很重要。曲线回答“训练过程中发生了什么”，而 evaluation 回答“固定测试记录上是否满足门槛”。例如 reward 变好不保证每个动作都合法，最后一轮 loss 更低也不保证它优于 best checkpoint。项目因此保留 best、last、manifest 和评估 JSON：它们共同组成可追溯证据，而不是让读者只根据一条曲线做部署决定。

四、验证、复现与排错

1.验证

先运行两套测试。这一步只确认已有代码、模型与依赖可以工作，不声称你已经重新训练出同一个 checkpoint。项目根目录执行：

Plain Text

```
npm.cmd test
npm.cmd run test:rl
```

成功信号是两个命令退出码均为 0。JavaScript 测试覆盖仿真、Exact 和浏览器 RL 逻辑；Python 测试覆盖数据、训练、评估和导出契约。若 test:rl 找不到 Python，检查 .venv\Scripts\python.exe 是否存在，并确认依赖没有装到系统 Python。

快速核对	命令或文件	你应确认的事实
浏览器模型	models/reinforcement-policy-ppo-v3.json	modelVersion 为 ppo-v3, seed 为 212
评估结果	artifacts/rl/evaluation-ppo-v3.json	600 条、合法率和覆盖率为 1
训练曲线	npm.cmd run rl:tensorboard	打开 http://127.0.0.1:6006
发布关联	checkpoint SHA 与 evaluation	selected checkpoint 与导出输入一致

Plain Text

```
Get-Content -Raw models\reinforcement-policy-ppo-v3.json
Get-Content -Raw artifacts\rl\evaluation-ppo-v3.json
npm.cmd run rl:tensorboard
```

TensorBoard 只读训练日志，不启动训练。它持续占用终端，停止时按 Ctrl+C。若 6006 被占用，结束旧 TensorBoard 进程或换端口；不要因为页面没有曲线就重新训练。

对照源码和制品时，只需先认识这一组最小文件地图：

- index.html 与 src/main.js：页面结构和启动入口。
- src/simulation.js、src/combat.js：仿真时间、事件和本地战斗。
- src/reinforcement-problem.js：Exact 与 RL 共用的问题、候选和 17 维特征契约。
- src/reinforcement-exact.js、src/reinforcement-policy.js：两种增援选择方式。
- src/planner-controls.js：默认 Exact、RL 切换和 PPO-v3 模型 URL。

- rl/environment.py、rl/model.py: Python 训练环境和 Actor-Critic 网络。
- rl/train.py、rl/evaluate.py、rl/export.py: 训练、门禁评估和浏览器导出。
- models/reinforcement-policy-ppo-v3.json: 页面当前加载的正式模型，实验时不要覆盖。
- artifacts/rl/evaluation-ppo-v3.json: 当前 600 条发布评估证据。

新手排查时不要从 300 多个 artifact 文件开始翻。先从页面状态确定问题属于启动、仿真还是模型，再沿上面入口查到配置、实现和制品。训练 run 中的 manifest.json、best/last checkpoint 与 TensorBoard 日志是一组证据，不应只留下一个 .pt 文件。

2. 数据生成

下面链条把所有新数据写到 repro 名称，不会覆盖正式的 train-v3.jsonl、validation-v3.jsonl、test-v3.jsonl、正式 checkpoint 或正式浏览器模型。数据生成会覆盖同名 JSONL，因此第一次运行前确认路径不存在；中断后应改用新的 repro 名称，不要把半成品当可信数据。

Python

```
npm.cmd run rl:data -- --output artifacts\rl\datasets\train-v3-repro.jsonl --count 400 --seed-start 1100000 --min-red 2 --max-red 20 --min-blue 1 --max-blue 81 --source mixed --simulation-share 0.2
npm.cmd run rl:data -- --output artifacts\rl\datasets\validation-v3-repro.jsonl --count 150 --seed-start 1300000 --min-red 2 --max-red 20 --min-blue 1 --max-blue 81 --source mixed --simulation-share 0.2 --exclude-jsonl artifacts\rl\datasets\train-v3-repro.jsonl
npm.cmd run rl:data -- --output artifacts\rl\datasets\test-v3-repro.jsonl --count 600 --seed-start 1500000 --min-red 2 --max-red 20 --min-blue 1 --max-blue 81 --source mixed --simulation-share 0.2 --exclude-jsonl artifacts\rl\datasets\train-v3-repro.jsonl --exclude-jsonl artifacts\rl\datasets\validation-v3-repro.jsonl
```

三份 manifest 的 sourceCounts 应分别是 320/80、120/30、480/120。--exclude-jsonl 防止 train、validation、test 问题指纹重叠。这里首先是方法复现：相同数据规模、算法和规则；没有固定 Git revision 与完整依赖锁时，不应承诺跨电脑逐字节重现正式模型。

每类输出承担的角色不同。JSONL 保存问题、候选和 Exact 专家动作；旁边的 manifest 记录生成参数与来源分布。训练 run 保存 best/last checkpoint、训练 manifest 和 TensorBoard 日志。pointer 只保存“当前选中了哪个 checkpoint”，evaluation JSON 才保存候选指标、门禁结果、selected checkpoint 和 hash。浏览器

JSON 是最后的推理权重，不包含可继续训练所需的优化器状态。

因此不能只保留最后一个模型文件。若 `evaluation` 指向的 `checkpoint`、`hash` 或模型版本与导出输入不一致，导出器会拒绝继续；这正是在阻止读者把不同实验的证据拼在一起。实验完成后至少保留数据 `manifest`、训练 `run`、`pointer`、`evaluation` 和导出模型 `hash`，才能说明结果来自哪条链。

3.训练

下面示例用正式流程的三个 BC 种子和三个 PPO 种子，结果全部写入 `*-repro-v3*`。训练 `run` 的受管文件不会覆盖已有 `run`；若目录已存在，换新后缀，不要删除不认识的实验目录。

Plain Text

```
foreach ($seed in 111,112,113) {
  npm.cmd run rl:train -- imitate --train-jsonl
artifacts\rl\datasets\train-v3-repro.jsonl --validation-jsonl
artifacts\rl\datasets\validation-v3-repro.jsonl --output-dir
"artifacts\rl\runs\bc-repro-v3-$seed" --seed $seed --epochs 3 --
batch-size 64 --learning-rate 0.0001
}
npm.cmd run rl:evaluate -- select-run --runs-dir
artifacts\rl\runs --prefix bc-repro-v3- --output-checkpoint-file
artifacts\rl\runs\selected-bc-repro-v3.txt
```

`select-run` 会原子更新指定的 `pointer` 文件，内容是一行选中的 `checkpoint` 路径。它与训练 `run` 的保护不同：想保存上一次选择结果时，先换一个 `pointer` 文件名或备份旧文件。单个种子可以帮助学习命令，但不能支持“三个种子择优”的结论。

Plain Text

```
$bcCheckpoint = (Get-Content -Raw artifacts\rl\runs\selected-bc-
repro-v3.txt).Trim()
foreach ($seed in 211,212,213) {
  npm.cmd run rl:train -- ppo --checkpoint $bcCheckpoint --train-
jsonl artifacts\rl\datasets\train-v3-repro.jsonl --validation-
jsonl artifacts\rl\datasets\validation-v3-repro.jsonl --output-dir
"artifacts\rl\runs\ppo-repro-v3-$seed" --seed $seed --updates 20 -
-episodes-per-update 16 --ppo-epochs 2 --batch-size 64 --learning-
rate 0.00001
}
npm.cmd run rl:evaluate -- select-run --runs-dir
artifacts\rl\runs --prefix ppo-repro-v3- --output-checkpoint-file
artifacts\rl\runs\selected-ppo-repro-v3.txt
```

接着分别评估“自动选最优候选”和“明确只部署 PPO”两种目的。`select-deploy` 会拒绝已存在的 `metrics` 或 `pointer` 输出，避免把不同实验的证据混在一起。若要重跑，使用新文件名，而不是覆盖旧 `evaluation`。

Plain Text

```
npm.cmd run rl:evaluate -- select-deploy --candidate-file
artifacts\rl\runs\selected-bc-repro-v3.txt --candidate-file
artifacts\rl\runs\selected-ppo-repro-v3.txt --dataset
artifacts\rl\datasets\test-v3-repro.jsonl --output-checkpoint-file
artifacts\rl\runs\selected-deploy-repro-v3.txt --output-metrics
artifacts\rl\evaluation-repro-v3.json
npm.cmd run rl:evaluate -- select-deploy --candidate-file
artifacts\rl\runs\selected-ppo-repro-v3.txt --dataset
artifacts\rl\datasets\test-v3-repro.jsonl --output-checkpoint-file
artifacts\rl\runs\selected-deploy-ppo-repro-v3.txt --output-
metrics artifacts\rl\evaluation-ppo-repro-v3.json
```

最后一条只以 PPO `pointer` 为候选，避免 BC 自动胜出后却手工改 `evaluation`。导出器会核对 `checkpoint`、`evaluation` 的 `selected checkpoint`、SHA-256 和质量门；不要手改 JSON 跳过这些检查。

Plain Text

```
$ppoCheckpoint = (Get-Content -Raw artifacts\rl\runs\selected-
deploy-ppo-repro-v3.txt).Trim()
npm.cmd run rl:export -- --checkpoint $ppoCheckpoint --metrics
artifacts\rl\evaluation-ppo-repro-v3.json --model-version ppo-
repro-v3 --output models\reinforcement-policy-ppo-repro-v3.json
Get-FileHash -Algorithm SHA256 $ppoCheckpoint,
artifacts\rl\evaluation-ppo-repro-v3.json, models\reinforcement-
policy-ppo-repro-v3.json
```

导出的实验模型不会自动被页面加载，也不应覆盖 `models\reinforcement-policy-ppo-v3.json`。只有完成评审并改动发布配置后，才可能把新模型变成产品默认；本教程的 `repro` 任务不授权这一步。

重跑实验前先判断目标路径的覆盖策略。数据脚本会覆盖同名 JSONL，所以最稳妥的做法是换 `repro-b` 等新后缀并重新核对行数；训练器保护已有受管 `run`，遇到冲突就换目录；`select-run` 可以原子更新 `pointer`，想保留旧选择必须先换名或备份；`select-deploy` 拒绝已有 `metrics` 和部署 `pointer`；导出模型也应使用新版本名。五类文件的行为不同，不能用“反正都是输出文件”概括。

如果只想学习流程，可以各跑一个 BC 和 PPO `seed`，并明确写成单次实验；若要复现正式的“三种种子择优”方法，就要完整运行 111/112/113 和 211/212/213，再执行两次 `select-run`。随机种子相同只约束随机过程，不能弥补依赖版本、数据字节或源码

revision 不同造成的差异。

4.排错

现象	先检查	正确处理
4173 端口占用	是否已有 npm.cmd run dev	结束旧服务器或改端口后重新打开页面
一直“配置加载中”	浏览器 Network 与 config 路径	通过服务器访问，不要双击 HTML
模型不可用或 fallback	模型文件、网络请求、历史原因	确认 ppo-v3 JSON; Exact 回退仍是安全结果
看不到蓝方	是否完成红方开局部署	等出生门打开后再等 5 秒
.venv 缺失	.venv\Scripts\python.exe	用 Python 3.12 在根目录创建环境
训练慢或输出冲突	CPU 日志、run/output 名称	观察 update; 使用新的 repro 名称

五、项目文件

[Air_Sim +DRL.zip]